v14: the outside participants opened up

> **Superseded by v16.0.** The shipped model is now `model/UFCEP6-0-3_Hospital_Patient_Pathway_v16.bpmn` : one hospital pool, three responsibility lanes holding nine team lanes, and the outside participants as black-box pools. Everything below describes the v14.0 fourteen-pool collaboration and is kept as the record of the release that was deployed and reviewed. What the shipped model contains is in 17-single-pool-operational-model.md.

> **Citations to files that are not in this tree.** An audit found 39 paths cited across 19 documents that no longer exist — the fourteen-pool model, the v1/v2 generators and the drawing scripts that went with them, and the Sprint 0 Node scaffold. Nothing was lost: all of it is in git history, and every citation is listed with what replaced it in 18-cited-files-no-longer-present.md.

Release v14.0. One file, one diagram: nine executable hospital processes and five external participants. No black-box pools remain.

The outside participants are drawn as **documentation pools**, not as executable processes. A supplier's pool shows the one step it performs and the hand-off in each direction, and it names the same job type the hospital's own service task already calls. The Java worker does the work; the pool shows where that work sits in the collaboration. The patient participant carries the referral entry point. Nothing in the collaboration is a pool that only names a party and explains nothing.

Revision note (supersedes the counts further down). An earlier build of v14 made all six outside participants executable, including a referring-organisation pool, and that build was run and logged (`screenshots/run-external-participants-v14.log`). It is not the file in this release. The referring-organisation pool has been **removed**: a GP surgery that refers in sits outside a model that begins when the referral arrives. The remaining five outside pools are drawn, and none of them is deployed as a process. Rows below that describe the executable build are kept for the record and marked as superseded.

This document sets out what changed, why this is a correctness fix and not decoration, how it was checked, and what still has to be re-run on an engine before it can be presented as tested.

---

1. What was wrong with the black boxes

v7.0 (`model/UFCEP6-0-3_Hospital_Patient_Pathway_v13.bpmn`) drew the six outside participants as pools with no process inside:

pool	processRef	drawn connections
`P_ReferringOrg`	*none*	2 (referral in, information request out)
`P_Correspondence`	*none*	1 (dispatch sub-process → pool)
`P_PaymentProvider`	*none*	**0: the pool floated**

pool	processRef	drawn connections
`P_ExternalClinicalServices`	*none*	**0, the pool floated**
`P_Patient`	*none*	3 (attends, contacts, refund message)
`P_ExternalScheduling`	*none*	**0 (the pool floated)**

Two separate defects, and only one of them is cosmetic:

1. **Nothing inside.** A reader could see that the hospital hands work to a supplier but not what the supplier does with it. The four suppliers are described in the case study at the boundary: slots come back, the dispatch result comes back, the payment status, reference, date and amount come back, capacity is reported. Leaving them empty threw away information the case study does give.
2. **Three pools with no connection at all.** `P_PaymentProvider`, `P_ExternalClinicalServices` and `P_ExternalScheduling` were reached only from service tasks inside the hospital pools (`payment.process-transaction`, `external-resources.check-availability`, `scheduling.find-appointment-slots`). The dashed lines that used to document those calls were removed deliberately in v7.0 (`_drop_redundant_supplier_flows`, "eleven of them, together about 63,000 px of dashed line"), and nothing replaced them. Three boxes on a collaboration diagram that no line touches read as a modelling mistake, because they are one: a participant that exchanges no messages is not participating.

The logic itself was not wrong: the hospital's calls, job types and error codes are correct and match the case study. What was missing was the other half of each exchange.

2. What v14 draws

Each outside participant gets one small process: what arrives, the single step that does the work, and the answer that goes back. The supplier pools are drawn `isExecutable="false"`: they document the boundary, while the hospital's own service task and the Java worker perform the step. That keeps one worker and one job type behind both ends of the same hand-off.

pool	process id	the process	step's job type
Patient or representative	`patient-representative`	start → *confirm attendance* → end	*(events only)*
External Scheduling Service	`external-scheduling-service`	slot search requested → *search the diary* → *return the slots* → end	`scheduling.find-appointment-slots`
External Correspondence Service	`external-correspondence-service`	letter received → *print and dispatch* → *return the dispatch result* → end	`correspondence.dispatch-letter`
External Payment Service Provider	`external-payment-service`	payment request → *process the card transaction* → *return status and reference* → end	`payment.process-transaction`
External Treatment, Laboratory and Imaging Services	`external-clinical-services`	capacity check → *check capacity* → *return what is available* → end	`external-resources.check-availability`

Drawn hand-offs were added and one re-pointed, taking the drawing from 15 dashed lines in v7.0 to **20 in this release**:

direction	from	to
request	`OUT_Auto_FindSlots`	`SCH_Start_SlotSearch`
response	`SCH_Throw_SlotsReturned`	`P_OutpatientBookings`
request	`SUB_Secretaries_Dispatch`	`COR_Start_Dispatch` *(re-pointed from the pool)*
response	`COR_Throw_Result`	`P_MedicalSecretaries`
request	`TRT_Auto_ProcessPayment`	`PAY_Start_Transaction`
response	`PAY_Throw_Result`	`P_TreatmentBookings`
request	`TRT_Auto_CheckExternalResources`	`EXT_Start_CapacityCheck`
response	`EXT_Throw_Capacity`	`P_TreatmentBookings`

The referrer and the patient keep the lines they already had: their hand-offs (`referral.received` , `info-requested` , `outcome-sent` , `appointment.attended` , `enquiry.received` , `refund.delayed`) were already message flows into and out of the pool, and re-pointing them at the new internal events would have changed the existing drawing without adding information.

3. The two decisions that keep it honest

The suppliers' steps reuse the hospital's job types. `scheduling.find-appointment-slots` , `correspondence.dispatch-letter` , `payment.process-transaction` and `external-resources.check-availability` are four of the 35 job types the model already declares and the Java workers already subscribe to. The same worker therefore drives both ends of the same exchange. Nothing new has to be written, no job type or form is added, and the worker job-type table in `workers/README.md` stays true. The v14 job-type census is **35 distinct types, unchanged**.

The hospital side is not touched. Its twelve supplier-facing service tasks, its eight error boundary events and its nine error codes are exactly as they were, so a supplier that cannot answer is still `SCHEDULING_SERVICE_UNAVAILABLE` , `CORRESPONDENCE_SERVICE_FAILED` , `PAYMENT_PROVIDER_UNAVAILABLE` or `EXTERNAL_RESOURCE_UNAVAILABLE` on the hospital's own task, and every run recorded in `docs/03-test-record.md` still describes the nine hospital processes that are in this file. The two sides are the two ends of the same job type. The alternative, rewriting all twelve calls as message throw/catch pairs and deleting the eight boundary events, is §5.

4. How v14 was checked

Everything here is measured on `model/UFCEP6-0-3_Hospital_Patient_Pathway_v14.bpmn` (SHA-256 `7bd812ad…`), against a local **c8run 8.10.0-alpha5** with the Java workers built from `workers/` running.

check	command	result
Generates deterministically	`python3 tools/build_v2.py` twice	identical SHA-256 (`7bd812ad…`)
Internal consistency of the spec	`build_v2.py` 's own checks (gateway defaults, form bindings, cell collisions)	clean
Parses as BPMN + Zeebe extensions	`node tools/validate_model.js model/…_v14.bpmn`	`parse warnings 0 / unresolved refs 0 / dangling seq refs 0 / 14 processes, executable 9 / user tasks without form 0 / service tasks without job type 0 / RESULT: clean` *(re-run against this file; the earlier build read 15 / 15)*
Imports in the Modeler's own renderer	`tools/render_diagram.sh` (bpmn-js 17.11.1)	renders, `5,741 × 15,476`
Nothing scored was lost	`python3 tools/verify_preservation.py`	**70 of 70 pass**
Drawing quality	`python3 tools/analyse_layout.py model/…_v14.bpmn`	0 overlapping segment pairs, 0 diagonal px, **1** line through an unrelated shape (`MessageFlow_03` through `SEC_Throw_ClinicalError`), **94** crossings, **20** message flows, canvas 5,882 × 15,202
No outside participant is a floating pool	`verify_preservation.py` (§6b)	checked against the earlier build; the script still expects the removed referring-organisation pool, see `docs/02-status.md`
No outside participant is still a black box	`verify_preservation.py` (§6b)	all five have a `processRef` and a drawn hand-off
The nine hospital processes still deploy and run	`./tools/deploy.sh` , then `tools/demo_scenario.py` and `--exception`	9 process definitions + 36 forms deployed; happy path `incidents: none` at all three checkpoints; the unreadable pack caught by the boundary event and routed into the missing-information loop (`screenshots/run-*-v14.log`)
~~All six outside participants execute~~ **superseded**: the executable build is not this file	`tools/check_external_participants.py` against the second deployment	`referring-organisation` , `patient-representative` , `external-scheduling-service` , `external-correspondence-service` , `external-payment-service` , `external-clinical-services` ; all **COMPLETED, no incident** (`screenshots/run-external-participants-v14.log`)
The integrations really fire	same run	the referrer's throw of `referral.received` started a Medical Secretaries instance; the patient's throw of `appointment.attended` started a Consultants instance

Three new checks matter for this release, and each one would have caught the v7.0 defect: every outside participant has a process of its own; every outside participant is connected to another pool; and no two drawn message flows share a pair of elements.

What running it found that checking it did not

Three defects, all fixed, none of which any static gate could see:

1. **`documentation` was written after `extensionElements`.** The BPMN XSD wants `bpmn:tBaseElement` children in the order documentation, extensionElements. bpmn-moddle happily imports either order, bpmn-js renders either order, and the Camunda deployer rejects the wrong one: `cvc-complex-type.2.4.a: invalid content was found starting with element 'documentation'`. It was latent for the whole life of the project because no process carried process-level documentation until the six outside participants did. Fixed in `tools/bpmn_builder_v2.py`.
2. **Two supplier steps inherited an input contract they could not satisfy.** The supplier's step runs the same job type as the hospital's task, so it also inherits the worker's validation. `correspondence.dispatch-letter` rejects a dispatch with no recipients (`CORRESPONDENCE_SERVICE_FAILED`) and `payment.process-transaction` requires a payment reference and a charge amount. Inside the hospital process those are guaranteed upstream; inside the supplier process, driven with a bare request, they are not. The first run produced two incidents. Both steps now carry a `zeebe:ioMapping` that defaults the fields, using the value when the request supplies it: `=if paymentReference = null then "SIMULATED-PAYMENT-REFERENCE" else paymentReference`.
3. **The check script reused correlation keys.** Camunda 8 will not start a second instance on a message start event while an instance with the same correlation key is still active: the same duplicate suppression the model relies on for hand-offs (`docs/04-modelling-decisions.md` §2). The first version of the check published a fixed reference and reported "process did not start" for three processes. The check now uses a fresh reference per run, and the behaviour it tripped over is itself evidence that the design works.

Numbers

	v7.0	v14.0
Participants	15	**14** (the referring organisation was removed)
Black-box participants	6	**0**
Processes (all executable)	9	**15**
Lanes	14	**20**
User tasks / forms	39 / 36	39 / 36
Service tasks	47	**51** (47 hospital + 4 supplier-side)
Distinct job types	35	35
Messages	57	**65**
Boundary events / error codes	18 / 9	18 / 9
Start / end events	43 / 66	**49 / 72**
Drawn message flows	15	**20**
Canvas	5,332 × 14,176	**5,812 × 15,184**
Preservation checks	67	**70**

5. Deployment: what the batch ceiling actually does, and what was verified

The ceiling is real, and it was measured. Camunda 8 stores the whole BPMN resource inside every process definition record, so a process count multiplies the resource size. Against c8run 8.10.0-alpha5, with all fifteen processes of the earlier build executable:

```
Command 'CREATE' rejected with code 'INVALID_ARGUMENT':
Can't append entry: ... valueType=PROCESS ... with size: 405002 this would exceed
the maximum batch size. [ currentBatchEntryCount: 24, currentBatchSize: 4095764 ]
```

Nine process records fit in the 4 MB batch; the tenth does not. That is exactly the constraint that justified black boxes in v7.0 (`docs/04-modelling-decisions.md` §7.6), now with a number against it.

What ships. One file and one diagram. The hospital's nine processes are executable and deploy in one batch; the five outside pools are drawn but not executable, so they do not need process records of their own. The earlier build instead set `EXTERNAL_PROCESSES_EXECUTABLE = False` (`tools/spec_v2.py`) and wrote the six outside processes with `isExecutable="false"`:

deployment	file	result
the nine hospital processes + 36 forms	`model/…_v14.bpmn`	**deployed**, key `2251799813952610`, v2 of each process
~~the six outside participants~~ **superseded**	`model/…_v14_external-participants.bpmn` (from `tools/build_external_participants.py`)	generated only for the executable build; not part of this release

The other two ways out, neither needed here:

1. **Raise the limit.** Set `zeebe.broker.network.maxMessageSize` above 4 MB in the broker configuration, set `EXTERNAL_PROCESSES_EXECUTABLE = True`, rebuild, and all fifteen deploy in one batch. Cleanest if the environment allows it, because then the second file is unnecessary; not tested here.
2. **Split further.** The second file already is the split; nothing more is needed.

Still open, in order of what a reviewer would ask:

#	what	why it is not done
1	Screenshots of v14 in Operate, Tasklist and the Modeler	`screenshots/` still holds the v7.0 images for those; the v14 evidence is the eight run logs plus `docs/11` §1–4
2	The Tasklist authentication `401` (DEF-14) and the short job-type table in `workers/README.md` (DEF-16)	pre-existing defects in `docs/08-test-plan.md` §9, unchanged by v14

Nothing that runs is left unrun. Every path the model carries was driven on v14: the happy path, the unreadable pack, declined payment, confirmation lost, the two escalation rungs, compensation, and the outside participants, each with `incidents: none` (`docs/03-test-record.md` §J11–J15).

Two things this release deliberately does not claim:

- The supplier processes are still not triggered by the hospital's service tasks. In Camunda 8 one process cannot call another except by message, and the hospital's call is a job (`scheduling.*`, `payment.*`, …). Making the two genuinely consecutive (throw a request, wait for the response, keep the existing exception handling) is the change described in `docs/04-modelling-decisions.md` §1 as considered and deferred. It is the natural v15.
- The second deployment file is a deployment artefact, not the deliverable: the deliverable is one collaboration and one diagram, and the outside participants are drawn in it. The second file contains the same six processes, generated from the same spec, so the two cannot drift.